SECTION 3 · CHEAT SHEET

Work safely in the App, and the CLI in brief

Every prompt, command, shortcut and setting from this section, lesson by lesson. Prompts are word for word, as typed on screen.

3.1 Section 3 introduction

Concept lesson: nothing to type, run or set.

3.2 Browser and terminal in the side panel

PROMPT

```
Read the menu and the cafe profile, then suggest one Google Fonts pairing for the cafe's brand: a heading font and a body font, with one line on why each fits. Don't create any files.
```

Sent in a new conversation in the cafe project. The answer: **Space Grotesk** for headings, **Inter** for body text.

SHORTCUT `Ctrl+T`

Opens the side panel's picker: the review of changes, a project file, a terminal, or a browser.

SETTING Side panel picker › Browser › address bar › fonts.google.com

A real browser built into the App, right beside the conversation. Here Google's free font library.

SETTING Search Space Grotesk › its page › the License tab

Shows the font large, then answers the one question a business asks: may it be used commercially?

SHORTCUT `Ctrl+L`

Select text on the page first; a small menu offers Comment and Quote. Ctrl+L quotes the selection into the message box.

PROMPT

```
Does this mean the cafe can use Space Grotesk on its website for free? Answer in one sentence.
```

Typed after the quote. The agent: yes, it has an open source license, free even for business.

SHORTCUT `Ctrl+J`

Opens the terminal in the same panel, already inside the project folder. Press it again to hide the panel, and once more to bring it back as it was.

COMMAND

```
git --version
```

Checks that Git is installed. Git saves versions of a project; the next lesson is built on it.

3.3 Saving versions

PROMPT

```
Turn this project folder into a Git repository and save all the current files as
the first version, with the message Starting files.
```

No Git commands to type: the agent runs them and checks each result. The folder becomes a repository on the branch **master**, and the first commit holds all **29** files.

PROMPT

```
Create brand/fonts.md with the font pairing we chose: Space Grotesk for headings
and Inter for body text, with one line on why each fits. Don't save a version yet.
```

The file is created and the agent notes it is not committed yet, so we can look first.

SETTING

Overview menu › Review

The review pane: every change since the last saved version. A new file shows all green.

SETTING

Review pane: the view menu › Uncommitted or Agent Edits

Uncommitted: everything changed since the last commit, by anyone. Agent Edits: only what the agent changed in this conversation.

SETTING

Hover the file › the plus button

Stages the file: chosen for the next version. Its letter changes from **U** (untracked) to **A** (added).

SETTING

Commit › message: Add the brand fonts › Commit

A note confirms the commit on master, and the review pane is empty again.

COMMAND

```
git log --oneline
```

In the terminal tab: the saved versions, newest first, each with its short code. Add the brand fonts on top, Starting files below.

RESULT

No Push button yet

Push copies your versions to GitHub. The cafe project has no GitHub copy yet, so every version stays on this computer.

3.4 Try ideas safely

SETTING

New conversation: the selector under the message box › New Worktree

Local Checkout works in the project folder itself. **New Worktree** gives the agent its own copy, on a new branch, away from the main project.

PROMPT

```
Write two short launch taglines for the Indiranagar opening into brand/taglines.md,
each under ten words, and tell me which one you recommend.
```

A worktree conversation runs in the background. In the sidebar it gets its own row, with its branch **indiranagar_launch_taglines** underneath.

PROMPT

```
You stopped before writing anything. Please create brand/taglines.md with the two
taglines now.
```

The first turn stopped with no answer and no file. Saying plainly what we want fixed it.

COMMAND

```
ls brand
```

In a conversation on the main project, its terminal: only fonts.md. The taglines exist only in the worktree.

PROMPT

```
I prefer the second one. Keep only Where code meets chai: now open in Indiranagar
in the file, and save it as a version on this branch.
```

The file now holds that one line, committed on the worktree's branch.

PROMPT

```
Good. Now bring this tagline into the main project, on the master branch.
```

The agent merges the branch and reports a **fast forward**: master had not changed since the branch began, so Git simply moved it forward.

COMMAND

```
ls brand -Name  then  git log --oneline
```

On the main project: both files, fonts.md and taglines.md, and the tagline commit on top of the history.

3.5 The CLI, a quick look

COMMAND

```
agy
```

In a terminal opened in the cafe project folder. The same agent and sign in as the App; the welcome shows the version, the Google account, the model and the folder.

COMMAND

```
/model
```

The same list of models as the App. Down picks **Gemini 3.8 Flash** (every plan has it), Right sets the effort to high, Enter confirms.

PROMPT

```
Read the owner's brief in baseline/new_outlet_brief.md and write a one-page launch plan for the Indiranagar outlet into plan/launch-plan.md: the goal, what we build and in what order, and how we will know it worked. Keep it short enough to read in two minutes.
```

Typed straight at the prompt, Enter sends it.

SETTING

Permission prompt › the first choice › Enter

The default mode reads freely but asks before each command or file. The first choice allows just this one; the others allow every command like it, for this conversation or for good.

SETTING

File prompt: Tab › No › Tab › type what to change › Enter

Says no to the draft and tells the agent what to do differently.

PROMPT

```
Use weeks before opening instead of calendar dates.
```

A new draft comes back, counted in weeks before opening.

SHORTCUT

`F`

On the file prompt: opens the whole file to read before deciding. Esc closes it, then yes allows it.

SHORTCUT

`Shift+Tab`

Moves through the execution modes: **Accept edits** (edits files without asking), **Auto** (also runs safe commands), **Plan** (researches and plans only), and back to the default, which asks first.

3.6 Review and undo in the CLI

SHORTCUT `Shift+Tab`

Once, to Accept edits: file edits go through without asking.

PROMPT
```
For the opening week, set every price in baseline/menu.csv to 0.
```

A bad change, on purpose. All **39** prices become zero.

COMMAND
```
/diff
```

All Changes: every file that differs from the last saved version. Enter opens a file: old lines first, then new. Right switches to **Per Turn**, the changes sorted by the message that made them.

COMMAND
```
/rewind
```

Lists your messages, each with the files it changed. Up picks the bad one, Enter goes back to just before it: the request returns to the prompt and the files go back too.

RESULT

/diff again shows only the launch plan

The menu is exactly as it was, every price restored.

COMMAND
```
/exit
```

Leaves the CLI and prints a command that continues this exact conversation.

COMMAND
```
agy  then  /resume
```

Lists past conversations, newest first. Typing filters the list (here **Indiranagar**); Enter opens one with everything done before.

PROMPT
```
In one sentence, what does our launch plan cover?
```

It remembers, and the bad request is gone from the history.